

Resumen Principios SOLID - Andrew Nicolay Prieto Mendoza

Los principios SOLID son recomendaciones clave para escribir código más legible, limpio, mantenible y escalable. Y que a nivel de software sea más robusto y eficiente.

Los **5 principios SOLID** son:

- S – Single Responsibility Principle (SRP)
- O – Open/Closed Principle (OCP)
- L – Liskov Substitution Principle (LSP)
- I – Interface Segregation Principle (ISP)
- D – Dependency Inversion Principle (DIP)

Vamos a dar una pequeña explicación más detallada:

- Principio de Responsabilidad Única (SRP):

- **“un módulo, clase o función debe tener una sola razón para cambiar”**

es decir, una única responsabilidad. El incumplimiento de esto lleva a componentes que realizan múltiples tareas, como hacer peticiones API, filtrar datos y renderizar UI, lo que dificulta las modificaciones ya que esto puede terminar en clases y microservicios saturados que **OBVIAMENTE** van a saturar nuestro programa. Al aplicar SRP, se segregan las responsabilidades en diferentes servicios o componentes (se segmentan las responsabilidades) facilitando el testeado y el mantenimiento.

- Principio Abierto/Cerrado (OCP): Aquí dice que el software debe estar abierto para extensión, pero cerrado para modificación. Esto significa que se puede añadir nuevo comportamiento sin alterar el código existente que ya funciona. Obviamente usar cosas como if o switch anidadas para nuevos tipos de flujo viola este principio ya que agregar nuevas funcionalidades solo hace el código más extendido y complejo, mientras que un diseño basado en objetos o mapeos de funciones permite añadir nuevas funcionalidades sin modificar el código principal.
- Principio de Sustitución de Liskov (LSP): Las subclasses o implementaciones deben poder sustituir a sus clases base o interfaces sin romper el comportamiento

esperado. Básicamente, este principio permite que se puedan reemplazar instancias de claves derivadas SIN tocar las instancias de la clase base, lo que permite extender el software sin errores en su código base.

- Principio de Segregación de Interfaces (ISP): Básicamente es un “No obligues a alguien a usar cosas que no necesita”. Usar interfaces "gigantes" donde muchas propiedades son opcionales porque no todas las implementaciones las necesitan. Si uno se pone a crear una “interfaz” muy grande, con muchas funciones y responsabilidades distintas, cualquier clase que quiera usar una pequeña parte de eso queda obligada a aceptar todo el paquete completo. La solución es crear interfaces más pequeñas y específicas, asegurando que cada componente o servicio solo use lo que realmente necesita, lo que también facilita el testeo.
- Principio de Inversión de Dependencias (DIP): Los módulos de alto nivel (aquellos que contienen la lógica principal del sistema) no deben depender directamente de módulos de bajo nivel (detalles técnicos o implementaciones concretas). Ambos deben depender de abstracciones. Esto significa que, en lugar de instanciar o utilizar directamente una clase concreta dentro de otra, se debe trabajar a través de una interfaz o contrato. De esta manera, las implementaciones específicas pueden cambiar sin afectar la lógica del sistema.

A lo último se menciona mencionando que es común encontrar código *legacy* que no sigue estos principios. Entonces toca aplicar algo llamado la "regla del *scout*" que básicamente es dejar el código mejor de lo que se encontró, implementando los principios SOLID poco a poco y priorizando siempre la entrega de valor limpio y correcto.